Integrated Navigation & Route Monitoring System

---

1. Understanding of the Problem Statement

Objective:
The objective of this assignment is to design a desktop navigation application that simulates the movement of a vessel along a planned route and provides real-time monitoring of navigation parameters.

Key Requirements:

- The application must display a vessel moving on a 2D chart-like view.
- The operator should be able to define, edit, and remove waypoints to manage the planned route.
- Vessel movement should be simulated in real-time with adjustable speed.
- The application should support orientation modes: North-Up and Heading-Up.
- Real-time navigation parameters, including position, speed, heading, distance to the next waypoint, and estimated time of arrival (ETA), should be displayed in an information panel.

Constraints / Key Points:

- Current implementation involves a single vessel simulation.
- Vessel movement is pixel-based and linear between waypoints.
- Waypoints can be added, moved, or removed during simulation.
- The system should be modular and extensible for future addition of multiple vessels or GPS data integration.

2. Approach and Design

Navigation Display:

- The application uses an HTML canvas for the 2D map view.
- The vessel, route legs, and waypoints are drawn dynamically using canvas APIs.
- Zoom and pan functionality is implemented using canvas transformations.
- Orientation modes are applied by rotating either the canvas or the vessel symbol depending on the selected mode.

Vessel Simulation:

- A vessel object stores current position, heading, and speed.
- Movement along the route is animated smoothly using `requestAnimationFrame`.
- Vessel heading is calculated dynamically using `Math.atan2` based on the next waypoint.

Route Management:

- Waypoints are stored in an array.
- Route legs are drawn connecting consecutive waypoints.
- The active route leg is highlighted visually.
- Waypoints can be added, moved, or removed interactively using mouse events.

Controls Module:

- Start/Pause buttons control the simulation.
- Speed slider adjusts vessel speed and updates the displayed value in real-time.
- Orientation buttons switch between North-Up and Heading-Up views.
- Mouse wheel allows zooming, and click-and-drag enables panning of the map.

Design Decisions & Justification:

- **Modular Design:** Canvas drawing, simulation, and controls are separated to support future extensibility.
- **Smooth Animation:** `requestAnimationFrame` ensures consistent updates and fluid movement.
- **Extensibility:** Adding multiple vessels or integrating real GPS data would require minimal changes, primarily to the vessel objects and coordinate system calculations.
- **User Interaction:** Interactive waypoint management allows flexible route adjustments during simulation.

The chosen approach was preferred over alternatives because it provides a **modular, extensible, and maintainable design** while ensuring smooth and accurate real-time simulation.

3. Solution Explanation

Step-by-Step Implementation:

1. **HTML Setup:**
    - Created `index.html` containing the canvas, control buttons, speed slider, and information panel.
2. **CSS Styling:**
    - Styled canvas, controls, and info panel for clarity and visual hierarchy.
3. **JavaScript Setup:**
    - Initialized canvas and 2D drawing context.
    - Defined a vessel object with properties for position (`x`, `y`), heading, and speed.
    - Created an array to store waypoints for route management.

4. **Drawing Function:**
    - Clears the canvas each frame.
    - Draws route legs connecting waypoints, highlighting the active leg.
    - Draws waypoints, marking the current waypoint differently.
    - Draws the vessel symbol with heading and a front indicator.
    - Updates the information panel with current position, speed, heading, distance to next waypoint, and ETA.
5. **Simulation Function:**
    - Updates vessel position incrementally along the current route leg.
    - Moves to the next waypoint when the current one is reached.
    - Computes heading dynamically to ensure the vessel points toward the target waypoint.
6. **Controls Implementation:**
    - Start/Pause buttons toggle simulation.
    - Speed slider changes vessel speed and updates the display value.
    - Orientation buttons toggle between North-Up and Heading-Up views.
    - Mouse wheel implements zoom; click-and-drag implements panning.
    - Mouse events allow adding, moving, or removing waypoints interactively.
7. **Helper Functions:**
    - `getWorldCoords(e)`: Converts mouse position to canvas coordinates accounting for pan and zoom.
    - `isMouseOverWaypoint(x, y)`: Detects whether the cursor is over a waypoint.

8. **Execution:**
    - Initial draw renders the map and vessel.
    - `requestAnimationFrame(update)` starts the simulation loop and continuously updates vessel position and navigation data.

Outcome:

- The vessel moves smoothly along the defined route.
- Waypoints can be managed interactively during simulation.
- Navigation data is updated in real-time in the information panel.
- Controls for speed, orientation, zoom, and pan function correctly.